Cache blocking of Matrix Multiplication and Convolution

Sparsh Mittal

Courtesy for some slides : NVIDIA

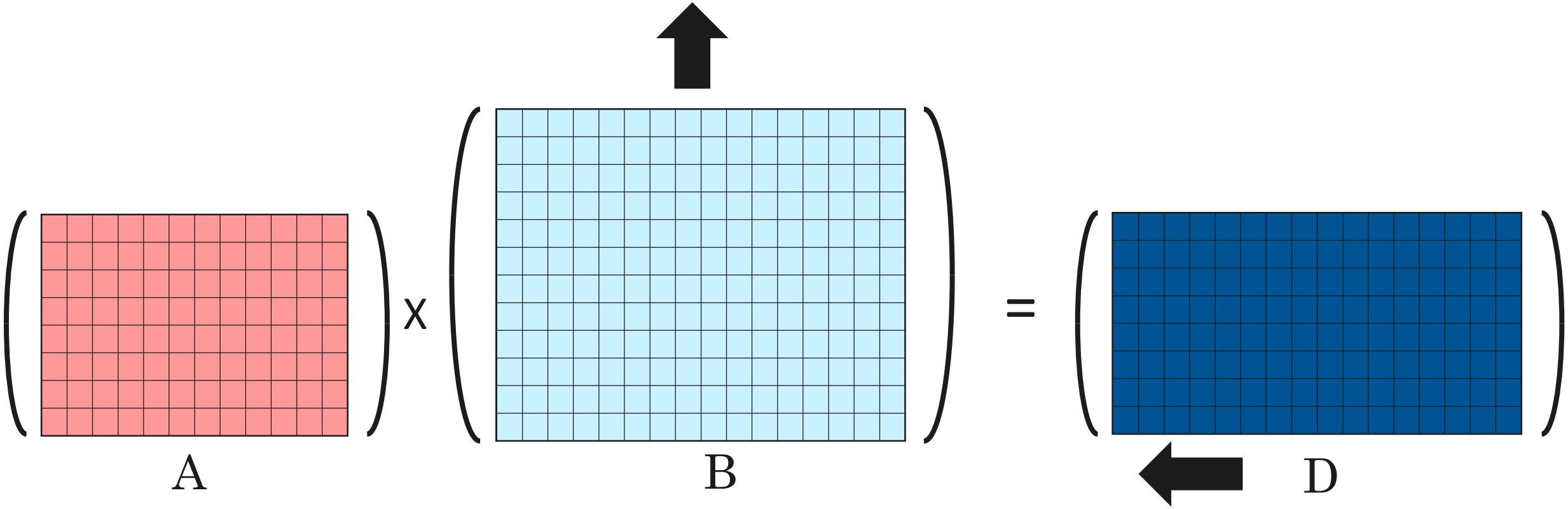
Matrix multiplication

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

$$= \begin{bmatrix} 1x10 + 2x20 + 3x30 & 1x11 + 2x21 + 3x31 \\ 4x10 + 5x20 + 6x30 & 4x11 + 5x21 + 6x31 \end{bmatrix}$$

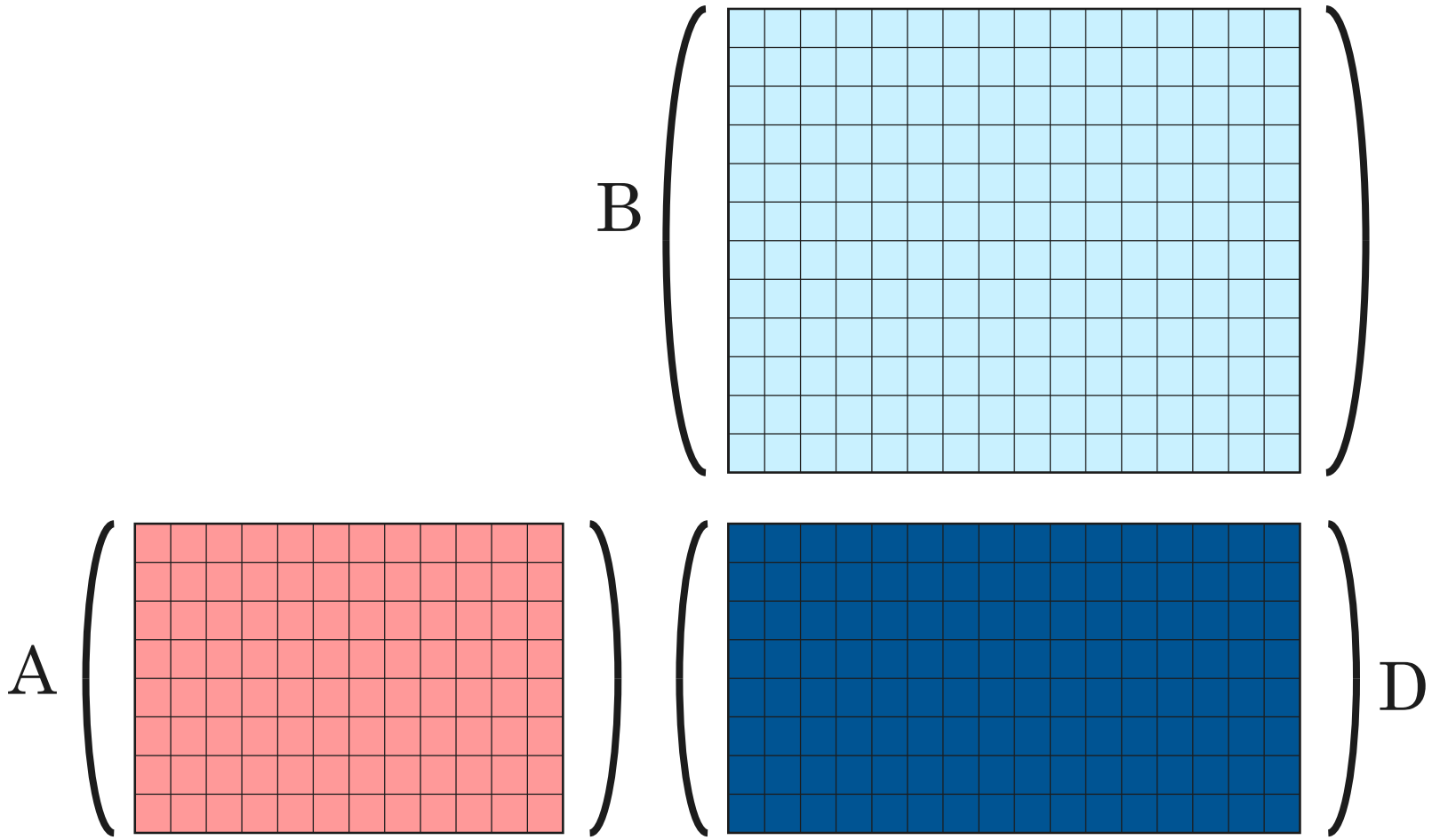
$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

matrix multiplication



Now, let us show these matrices in a slightly different way.
Let us push B upwards and D towards the left.

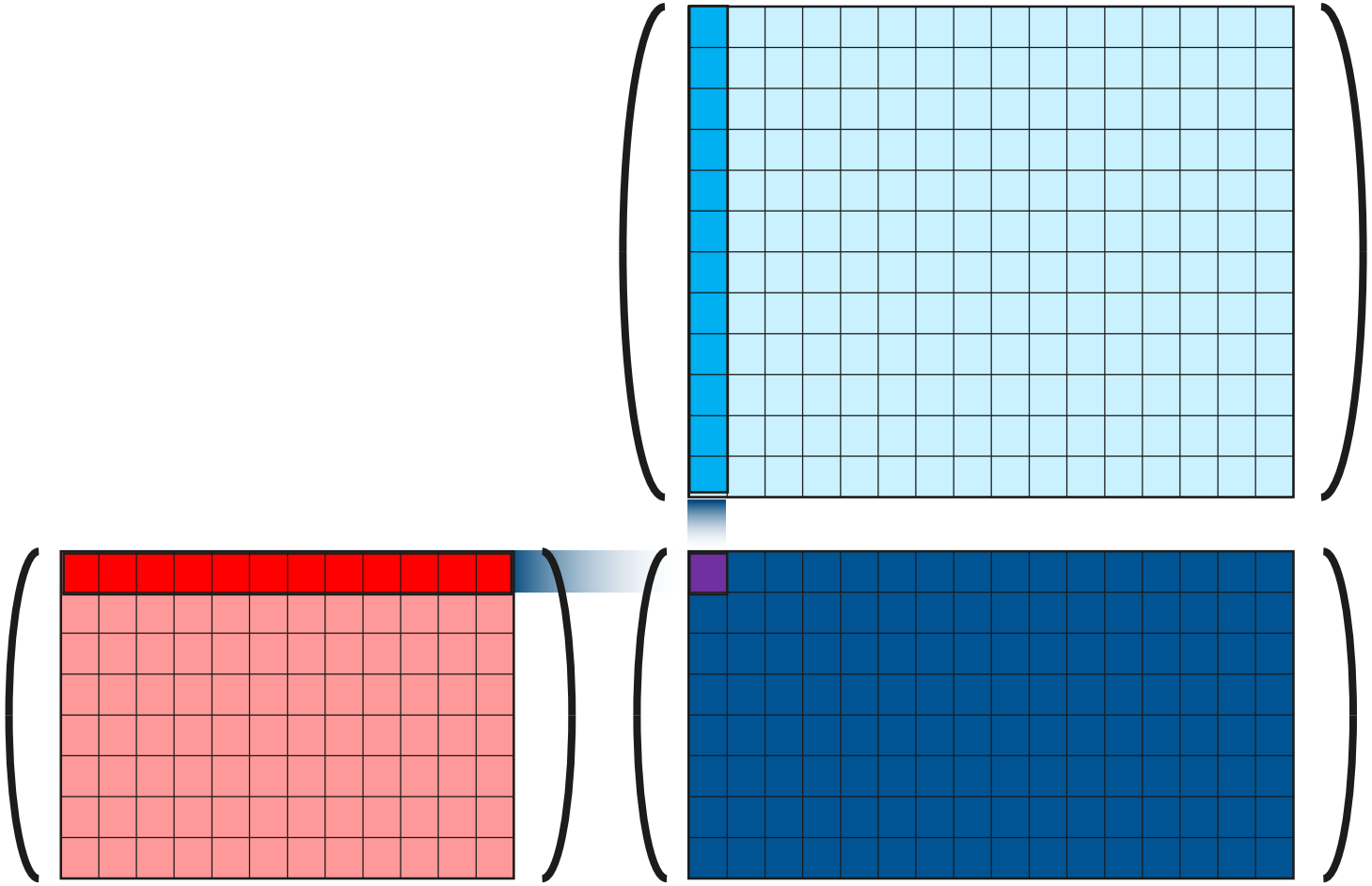
matrix multiplication



Courtesy : NVIDIA

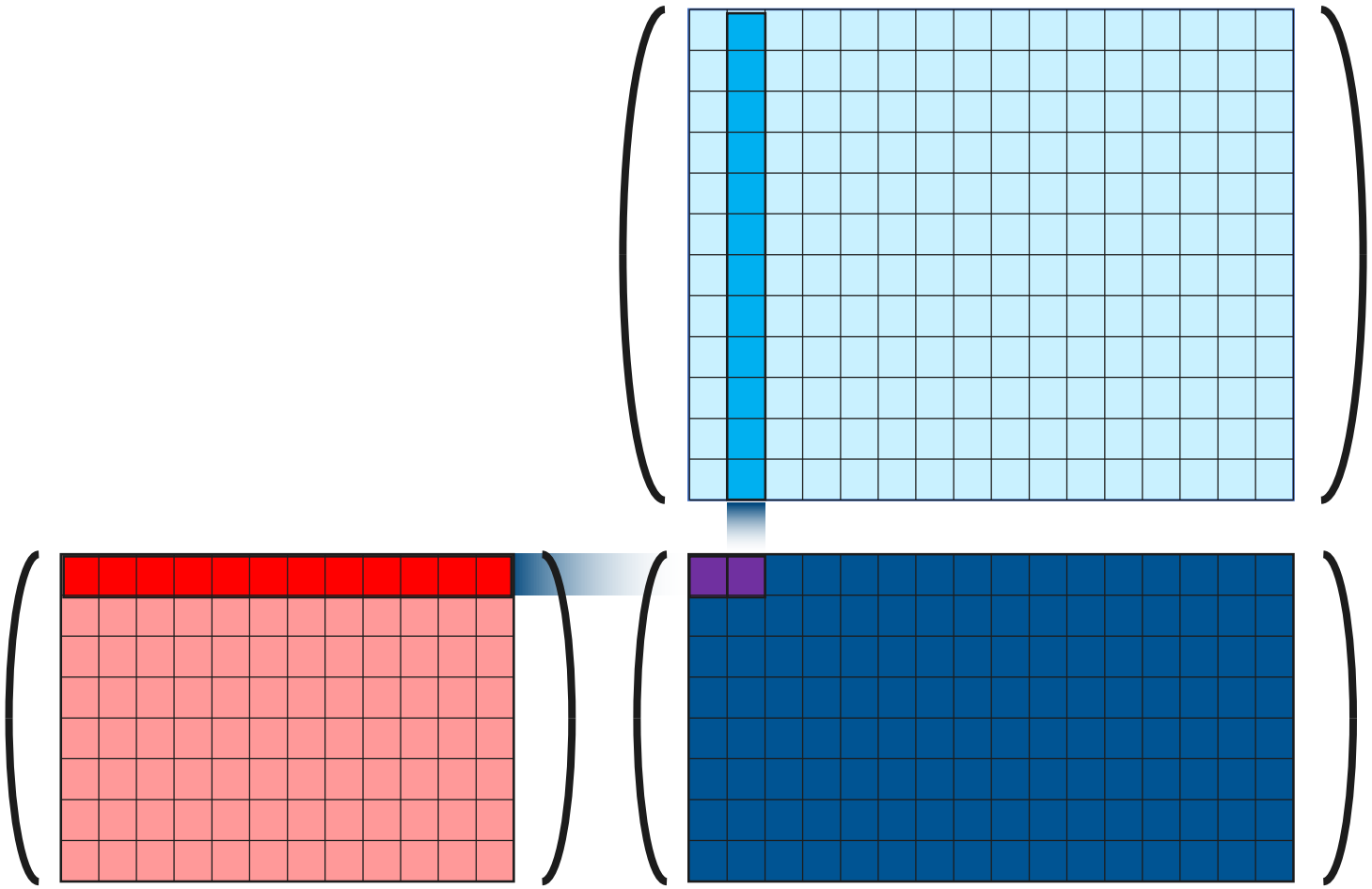
matrix multiplication

A naïve method



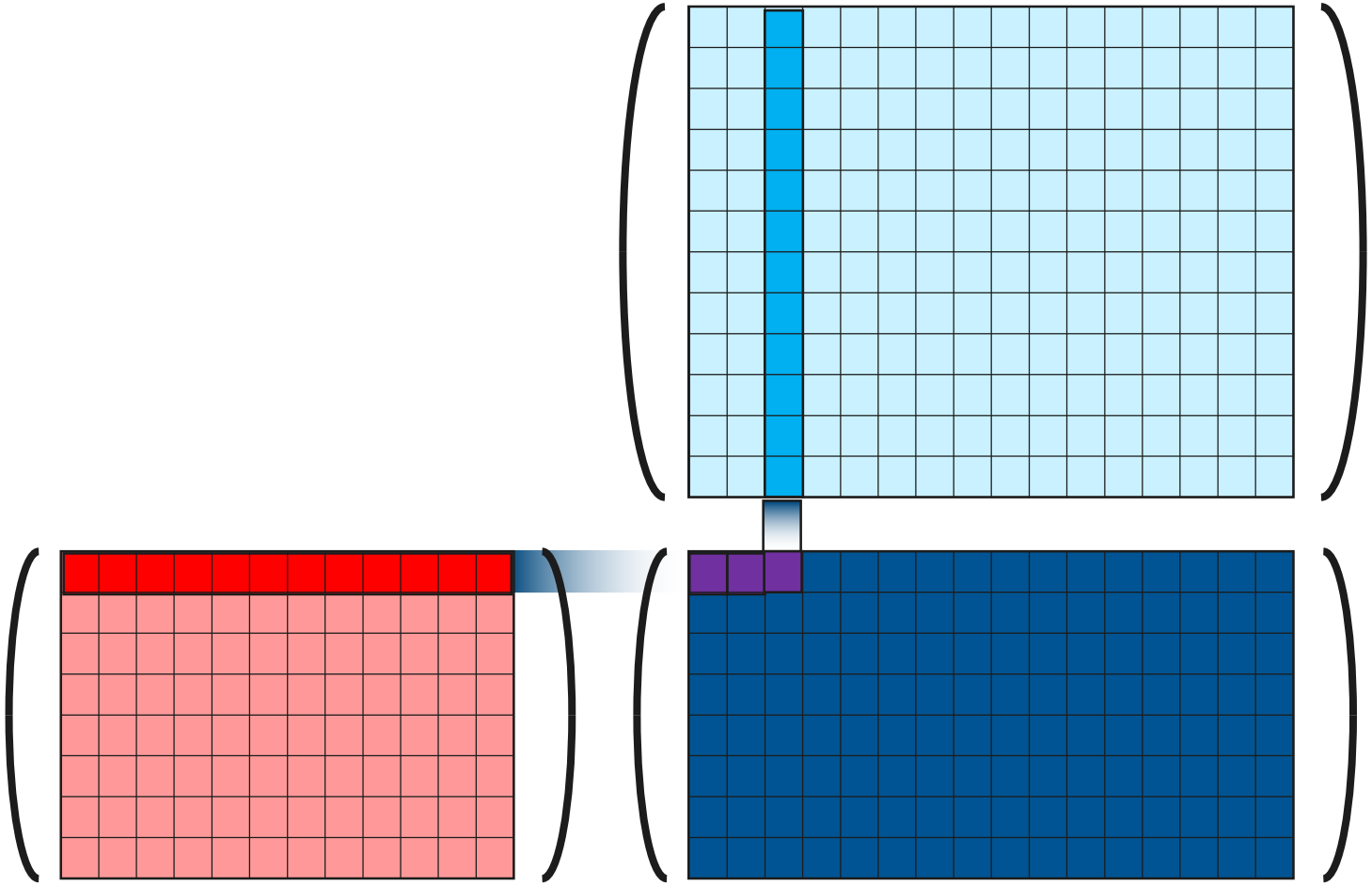
matrix multiplication

A naïve method



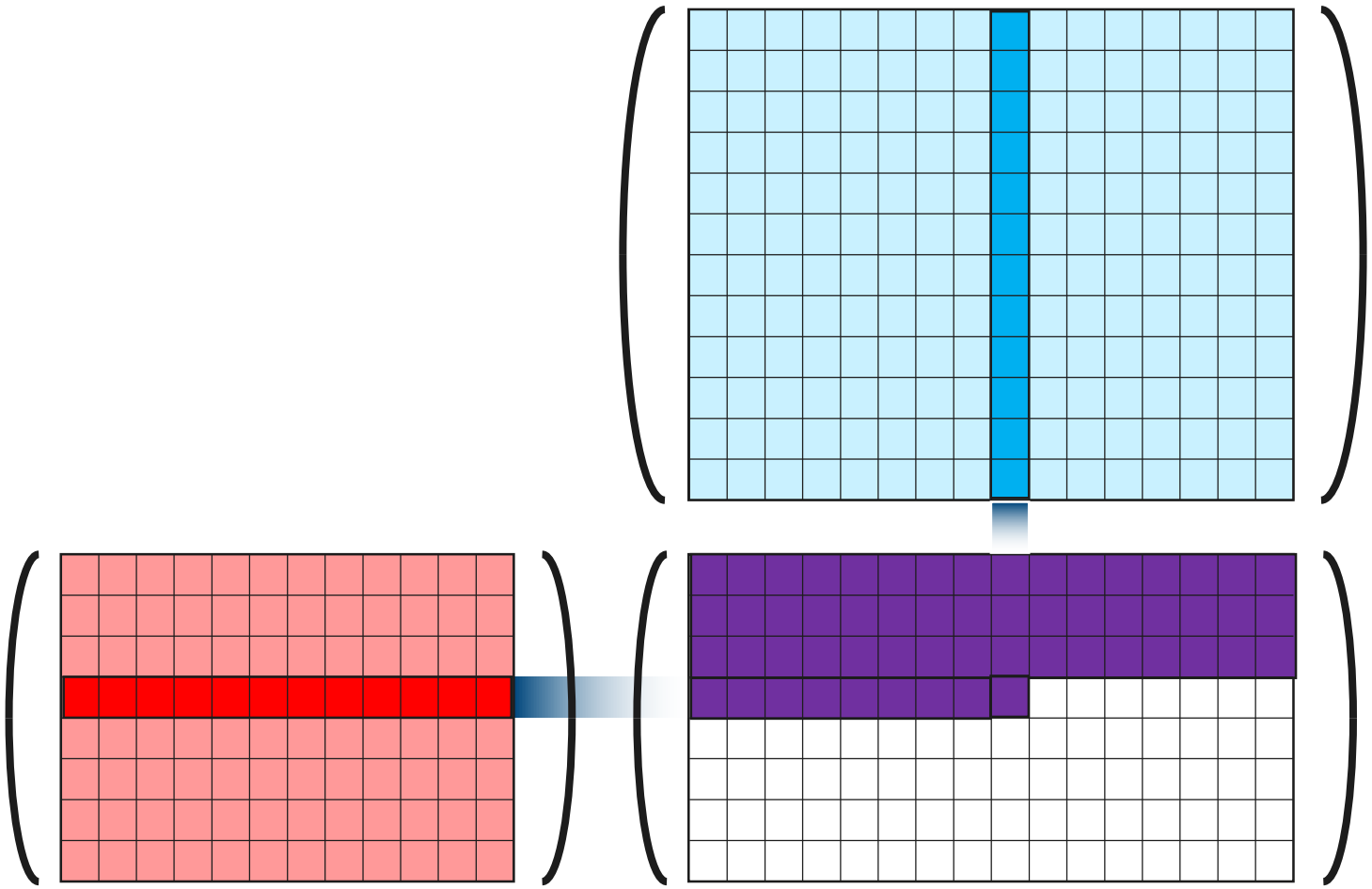
matrix multiplication

A naïve method



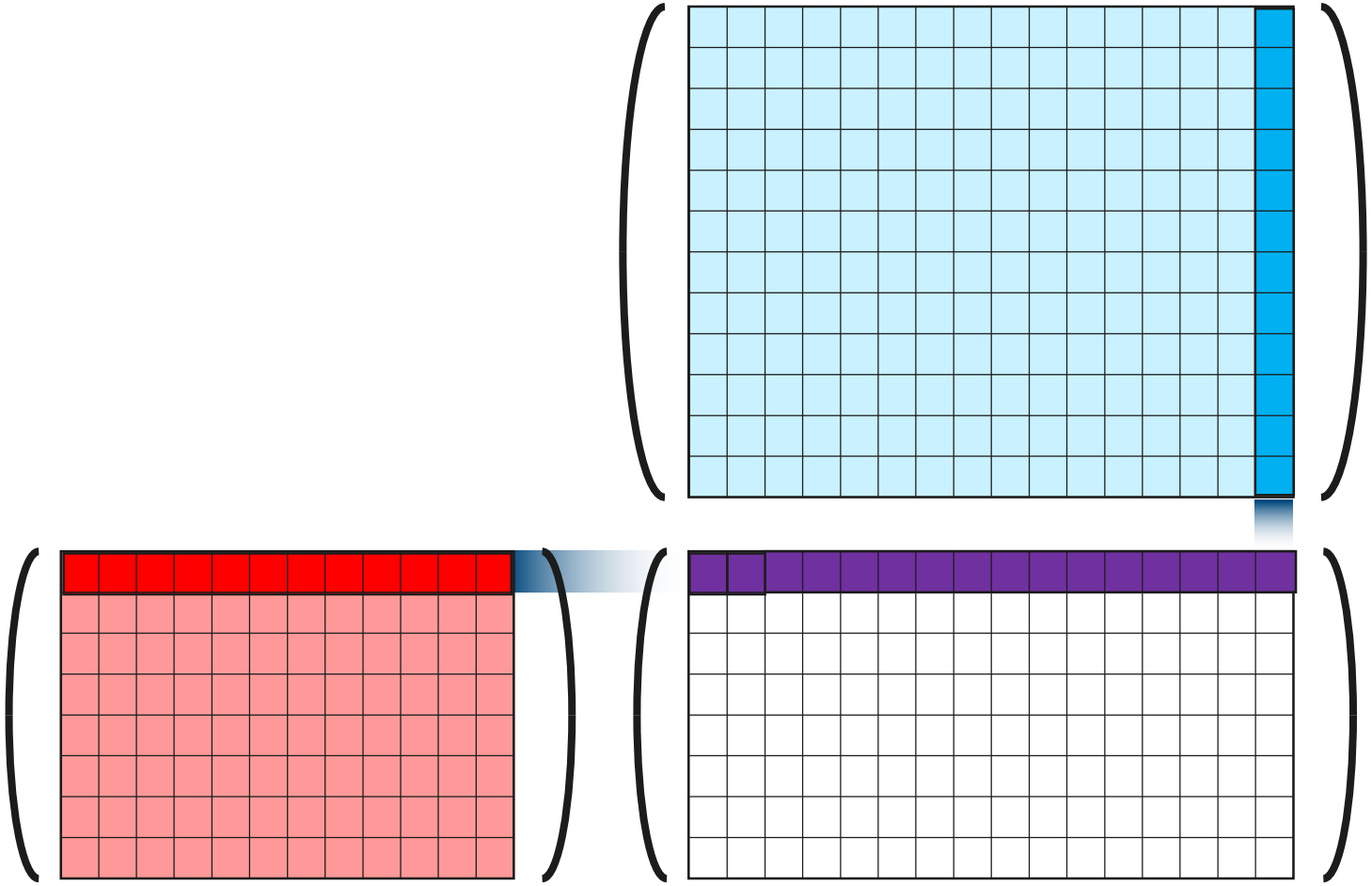
matrix multiplication

A naïve method



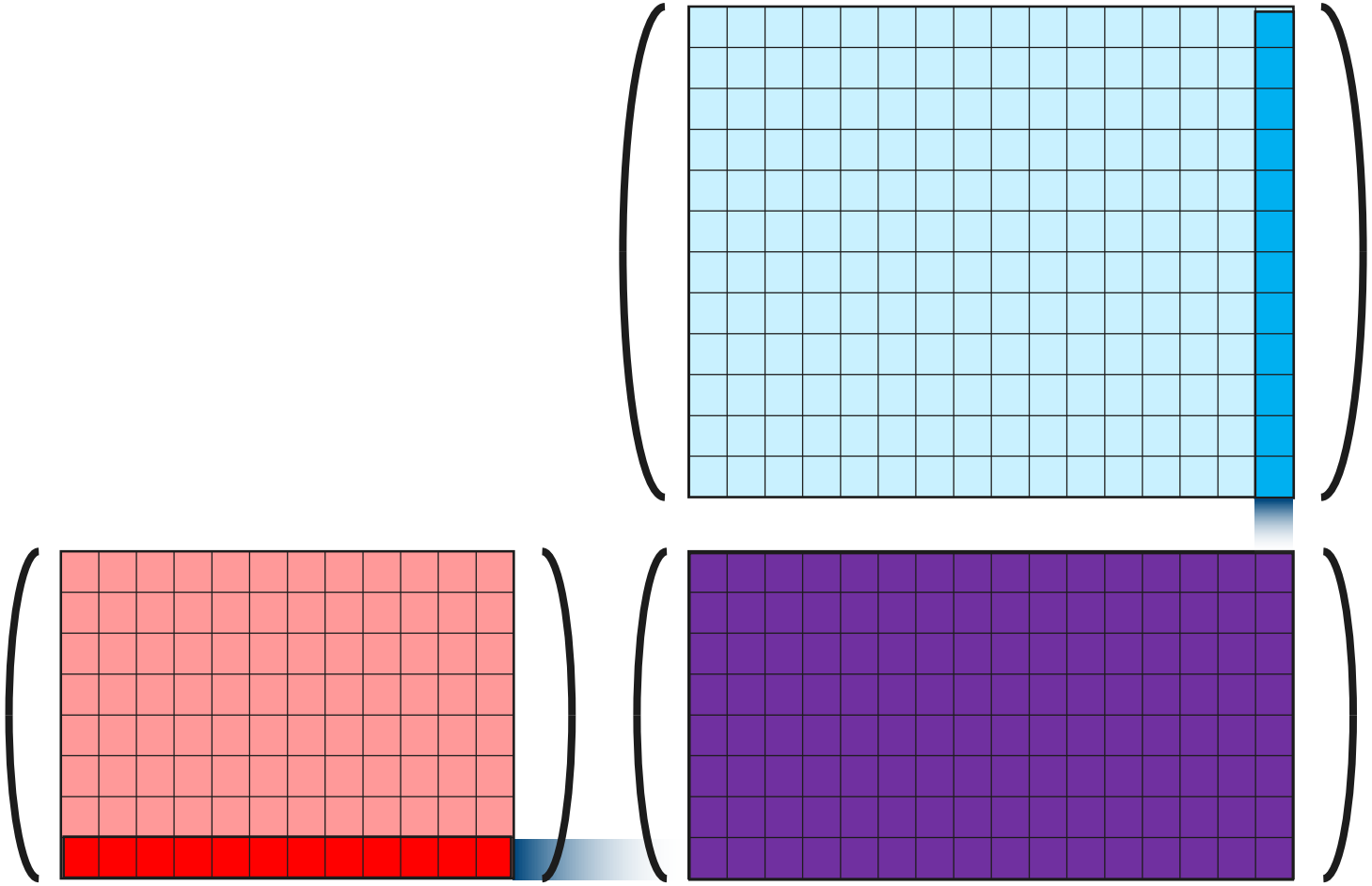
matrix multiplication

A naïve method



matrix multiplication

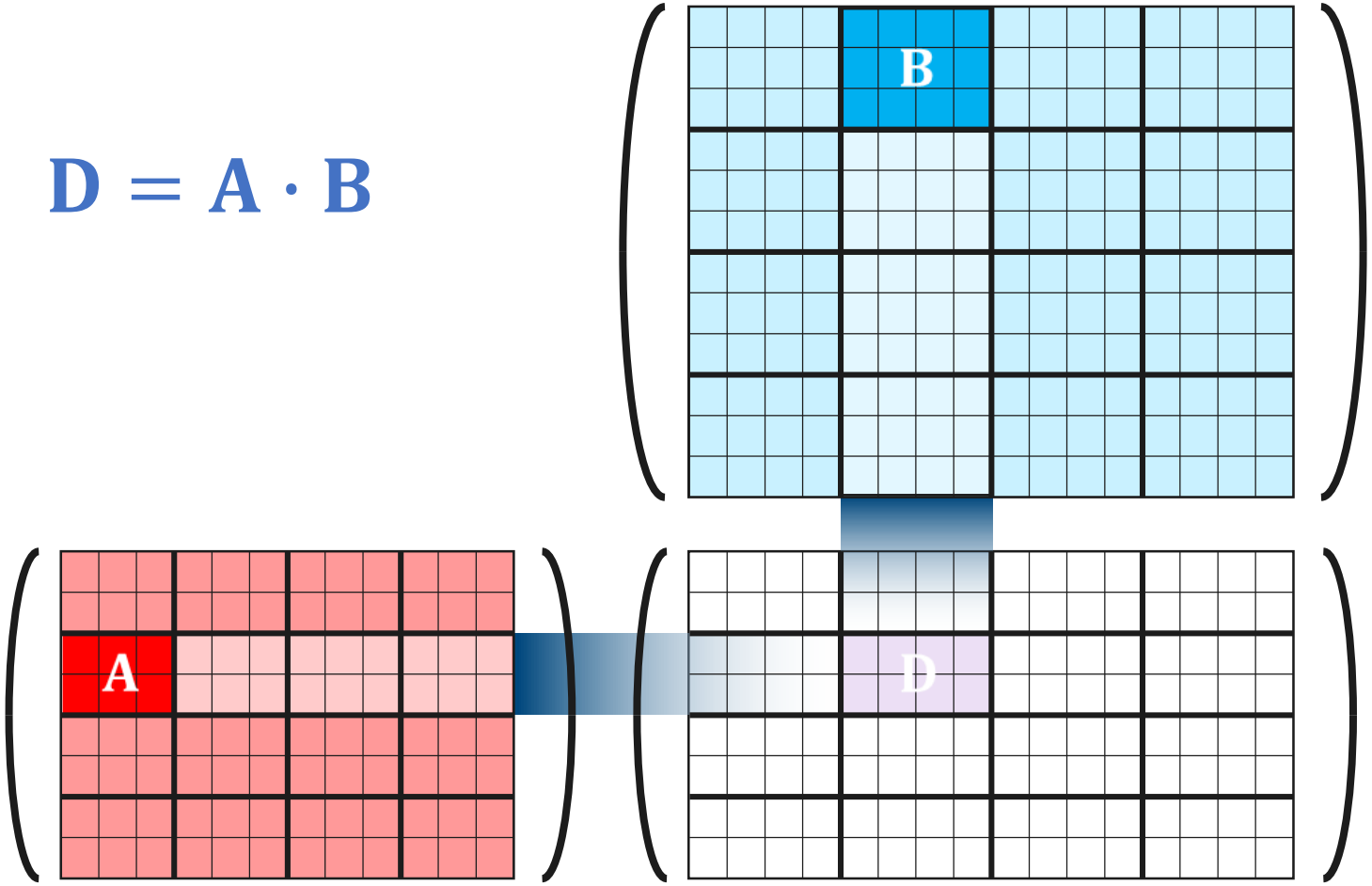
A naïve method



Tiled matrix multiplication

Memory efficient

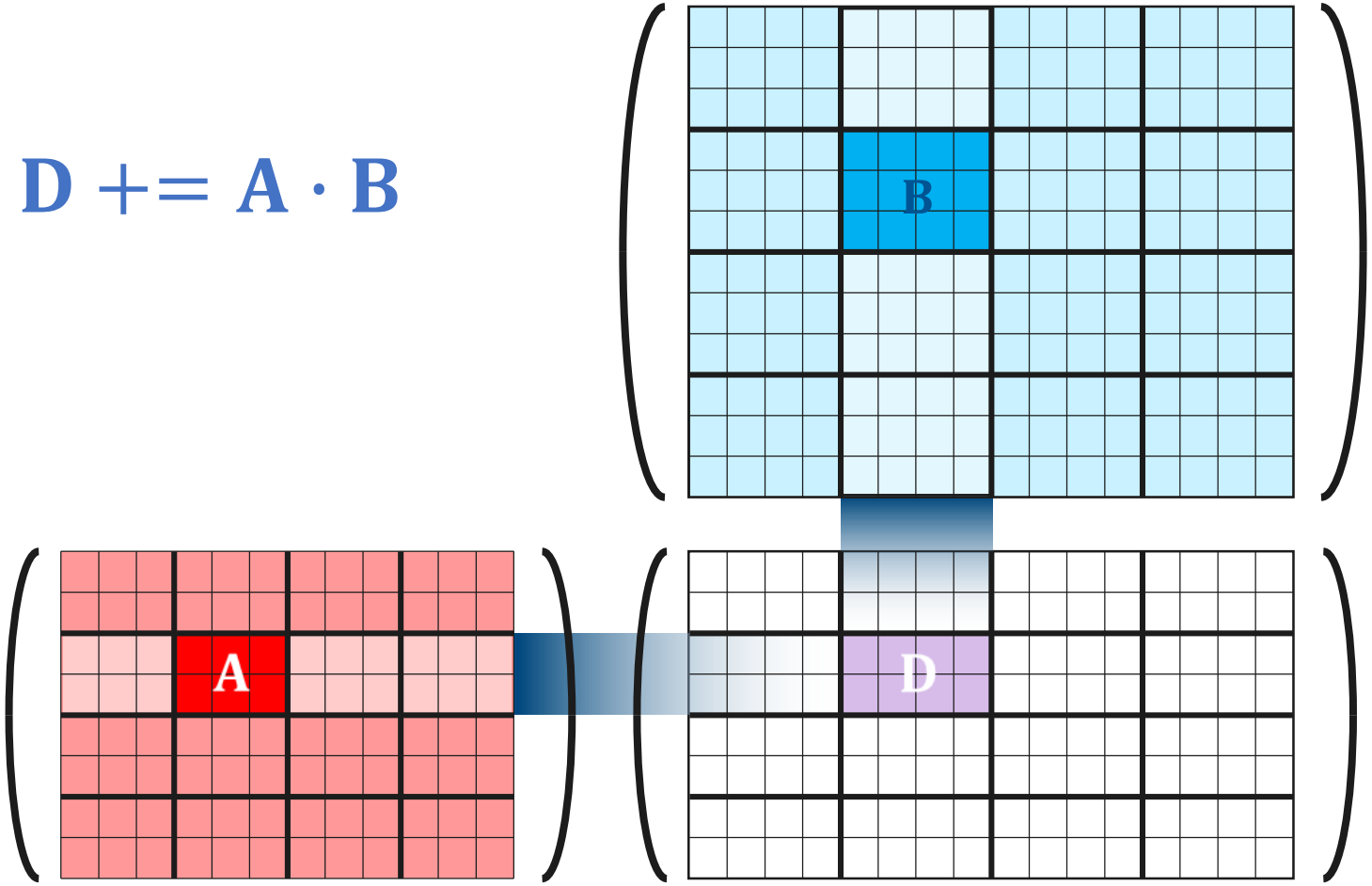
$$\mathbf{D} = \mathbf{A} \cdot \mathbf{B}$$



Tiled matrix multiplication

Memory efficient

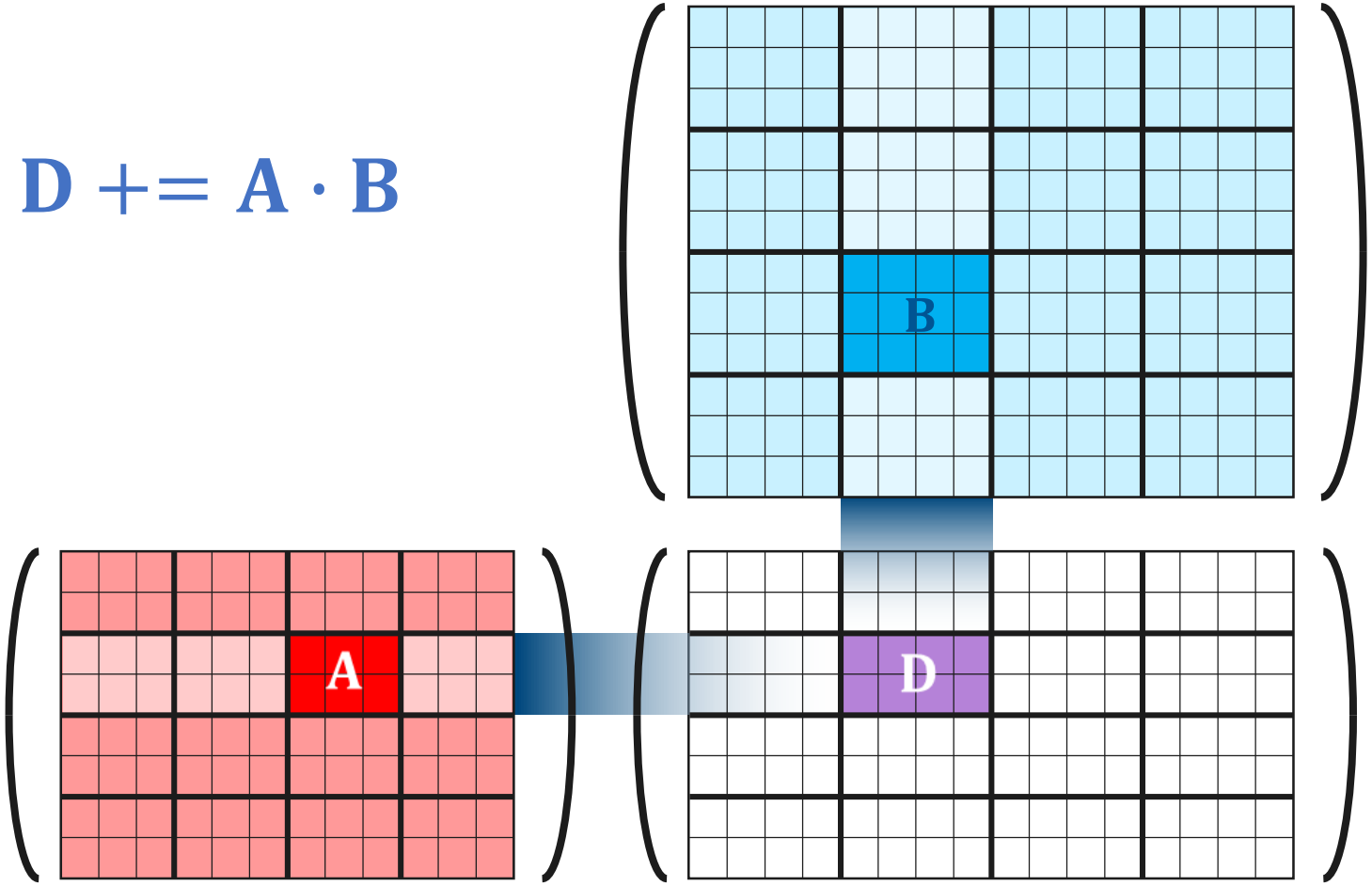
$$D \mathrel{+}= A \cdot B$$



Tiled matrix multiplication

Memory efficient

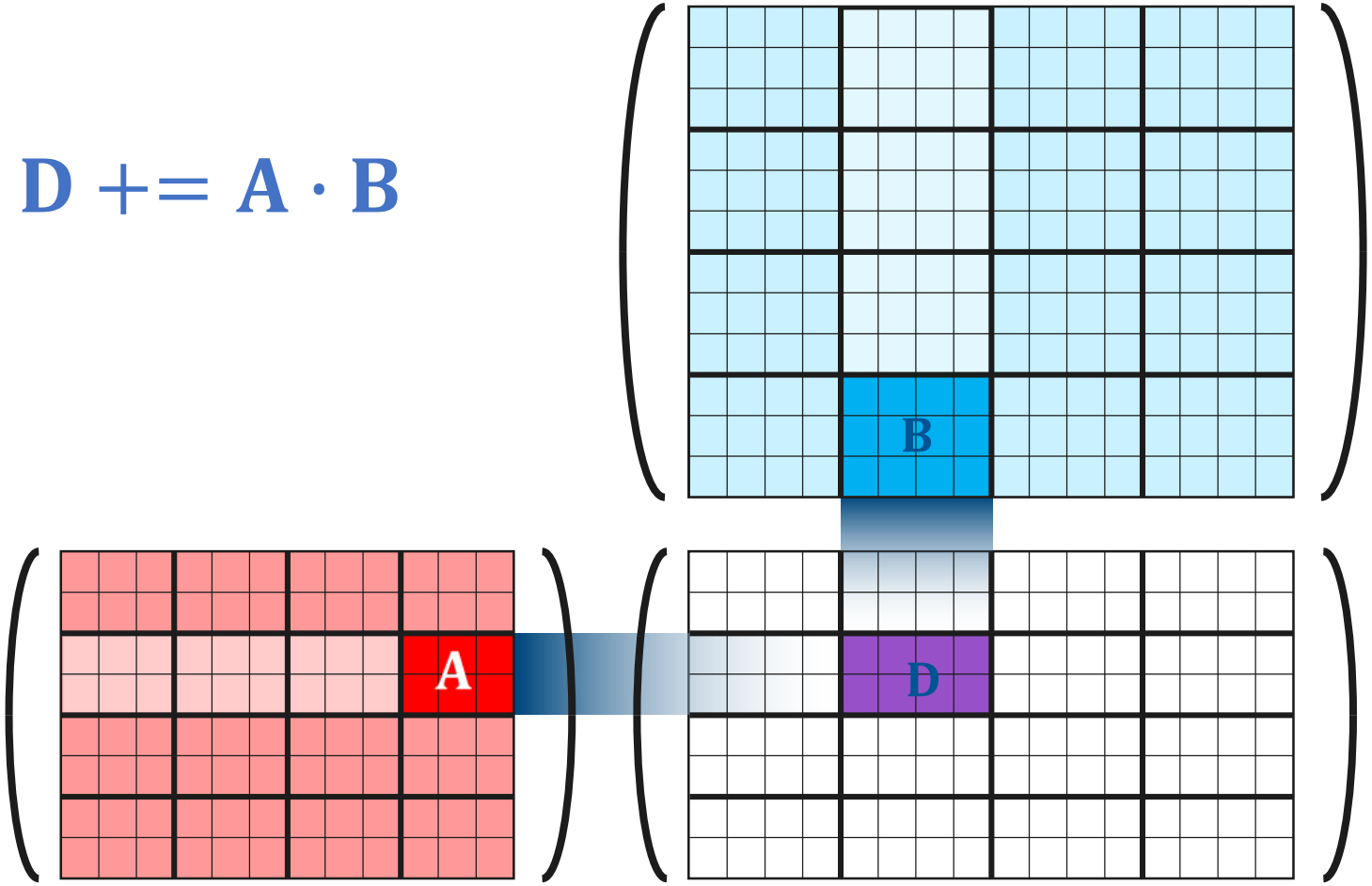
$$\mathbf{D} \mathrel{+}= \mathbf{A} \cdot \mathbf{B}$$



Tiled matrix multiplication

Memory efficient

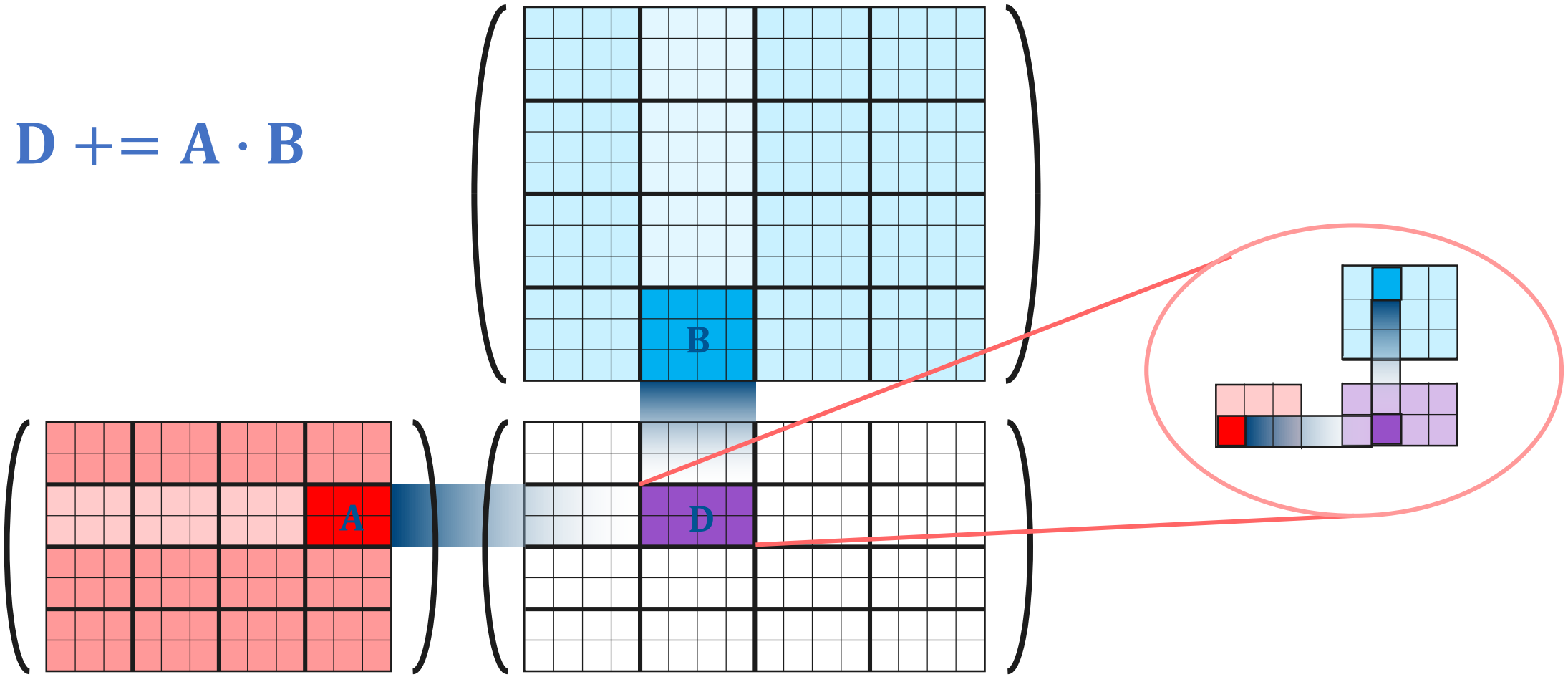
$$\mathbf{D} \mathrel{+}= \mathbf{A} \cdot \mathbf{B}$$



Tiled matrix multiplication

Memory efficient, hierarchical

$$\mathbf{D} \mathrel{+}= \mathbf{A} \cdot \mathbf{B}$$



Blocking of matrix multiplication

Slides based on: Computer Architecture Book (6th Edition, page 107) by Hennessey Patterson

Matrix-multiplication before blocking

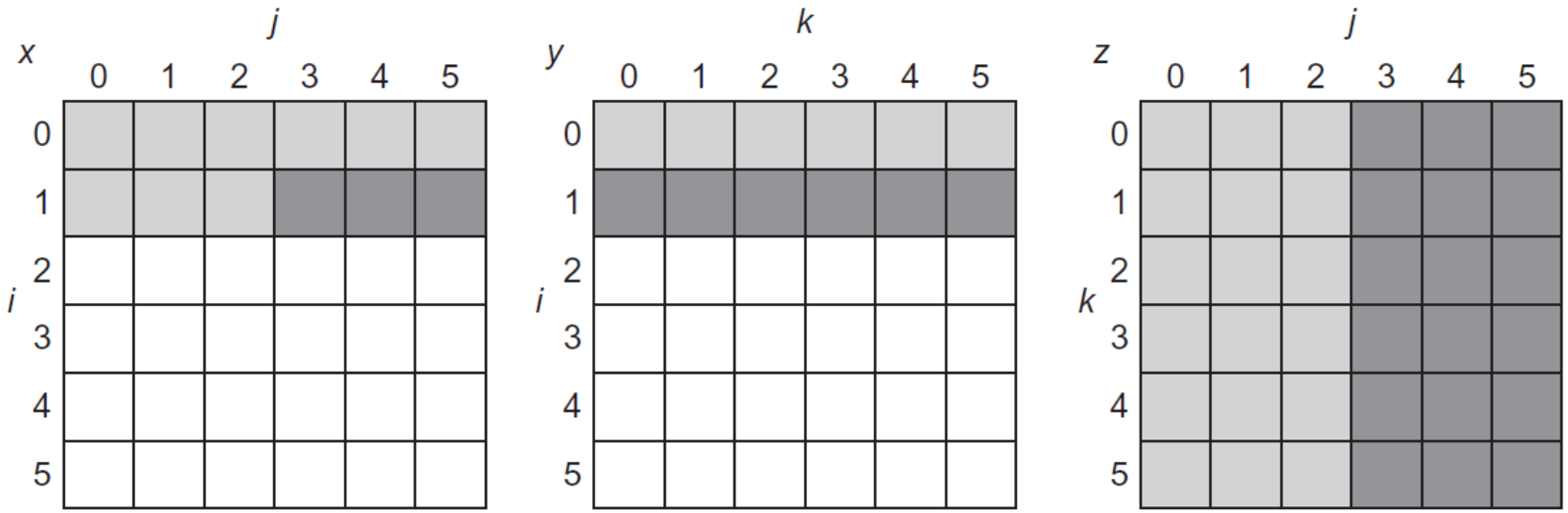
```
for (i = 0; i < N; i = i + 1)
    for (j = 0; j < N; j = j + 1)
        {r = 0;
         for (k = 0; k < N; k = k + 1)
             r = r + y[i][k]*z[k][j];
         x[i][j] = r;
        };
```

(Assume x is initialized to zero.)

If N=6, there are a total of 6*6*(6*2+1) = 468 accesses

Minimum misses = compulsory misses = 3*6*6 = 108

Snapshot of arrays when N=6 and i=1



The age of accesses to the array elements is indicated by shade:
white means not yet touched,
light means older accesses, and
dark means newer accesses.

The elements of y and z are read repeatedly to calculate new elements of x.

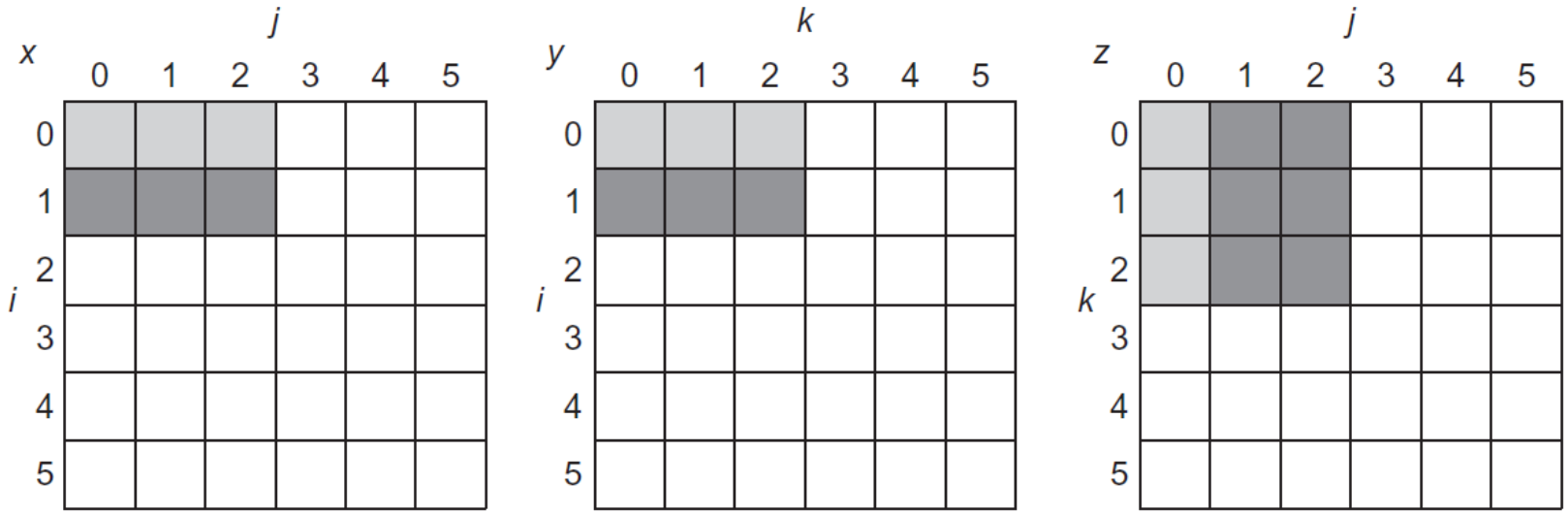
The variables i, j, and k are shown along the rows or columns used to access the arrays.

Matrix-multiplication after blocking

```
for (jj = 0; jj < N; jj = jj + B)
for (kk = 0; kk < N; kk = kk + B)
for (i = 0; i < N; i = i + 1)
        for (j = jj; j < min(jj + B,N); j = j + 1)
            {r = 0;
             for (k = kk; k < min(kk + B,N); k = k + 1)
                  r = r + y[i][k]*z[k][j];
             x[i][j] = x[i][j] + r;
            };
```

- We change code to compute on a submatrix of size B by B
- Two inner loops now compute in steps of size B rather than the full length of x and z. B is called the blocking factor.

Snapshot of arrays when N=6 and i=1



The age of accesses to the arrays x, y, and z when B=3.

Compared to original code, less elements are accessed.

Cache misses in blocked and unblocked cache

	Number of cache misses	
CacheSize	No blocking	Blocking
16	237	168
18	227	161
20	217	157
22	207	153
24	197	149
26	187	145
28	177	141
30	167	137
32	157	133
34	147	129
36	137	125
38	128	121
40	120	117
42	112	114
44	108	112
46	108	110
48	108	108

Blocking of convolution kernel

CONV layer before/after tiling

```
for ( row=0; row<R; row++)
 for ( col =0; col<C; col++)
  for ( to=0; to<M; to++)
   for ( ti =0; ti <N; ti++)



 for ( i =0; i<K; i++)
  for ( j =0; j<K; j++)
    Output_fmaps [to] [row] [col]
+= Weights [to] [ti] [i] [j] *
Input_fmaps[ti] [S*row+i] [S*col+j]
```

CONV layer code before tiling

Orig. loops

Tiled loops

Not-tiled loops

```
for ( row=0; row<R; row+=Tr)
 for ( col =0; col<C; col+=Tc)
  for ( to=0; to<M; to+=Tm)
   for ( ti =0; ti <N; ti+=Tn)
    for (trr=row; trr<(row+Tr); trr++)
     for (tcc=col ; tcc<(col+Tc); tcc++)
      for (too=to; too<(to+Tm); too++)
       for (tii=ti; tii <(ti+Tn)  ; tii++)
        for ( i =0; i<K; i++)
         for ( j =0; j<K; j++)
    Output_fmaps [too] [trr] [tcc] +=
    Weights [too] [tii] [i] [j] *
 Input_fmaps [tii] [S*trr+i] [S*tcc+j]
```

CONV layer code after tiling

CONV = Convolution